

E-Commerce Order & Payment Management System

Complete Java Full Stack Development Guide — Requirements, Architecture, Database, APIs, Development Stages, Testing and Deployment

Backend	Java 17+, Spring Boot, Spring Security, Spring Data JPA, Hibernate, JWT
Frontend	React.js, React Router, Axios, Context API or Redux Toolkit
Database	PostgreSQL
Testing	Postman, JUnit, Mockito, Spring Boot Test
Build & Tools	Maven, Git, GitHub, IntelliJ IDEA/Eclipse, VS Code
Optional	Docker, cloud deployment, sandbox payment gateway

Project objective: Build a realistic e-commerce application rather than a basic CRUD demo. The system should demonstrate authentication, authorization, product management, cart and checkout workflows, inventory, payments, transactions, exception handling, testing, and a React frontend.

Recommended scope: Start with two roles — CUSTOMER and ADMIN. Add seller/delivery roles only after the core system is stable.

1. Functional Requirements

Customer Features

- Register and log in securely.
- Browse, search, filter, sort, and paginate products.
- View product details and category information.
- Add products to cart, change quantities, and remove items.
- Manage multiple delivery addresses.
- Checkout and place orders.
- Make or simulate payment and view payment status.
- View order history and order details.
- Cancel eligible orders.
- Manage profile information.

Admin Features

- Secure admin login with role-based authorization.
- Create, update, deactivate, and manage products.
- Create and manage categories.
- View and update order status.
- Manage inventory and identify low-stock products.
- View customers and basic business statistics.
- View dashboard metrics such as users, products, orders, and revenue.

Non-Functional Requirements

- Use layered architecture and keep business logic in services, not controllers.
- Validate incoming requests and return consistent API errors.
- Protect admin and customer endpoints using Spring Security and JWT.
- Use database transactions for multi-step operations such as order creation.
- Use Git with meaningful commits and maintain a professional README.
- Write tests for critical services and business rules.

2. System Architecture

```
React.js
  ↓ HTTP / JSON
REST Controllers
  ↓
Service Layer
  ↓
Spring Data JPA / Hibernate
  ↓
PostgreSQL
```

Recommended package structure:

```
com.ecommerce
├── controller
├── service
├── impl
├── repository
├── entity
├── dto
├── mapper
```

```
■■■ security
■■■ exception
■■■ config
■■■ EcommerceApplication
```

Rule: Controller handles HTTP concerns, Service handles business logic, Repository handles persistence. Avoid putting business rules directly inside controllers.

3. Database Design

Initial tables/entities:

- users — customer/admin account information.
- roles and user_roles — authorization roles.
- categories — product categories.
- products — product details, price, image URL, active status.
- addresses — customer delivery addresses.
- cart and cart_items — active shopping cart and its products.
- orders and order_items — order header and individual purchased products.
- payments — payment amount, method, status, and transaction reference.
- inventory — available and reserved quantities.

Core Relationships

```
User 1 ■■■ * Address
User 1 ■■■ 1 Cart
Cart 1 ■■■ * CartItem
Product 1 ■■■ * CartItem
Category 1 ■■■ * Product

User 1 ■■■ * Order
Order 1 ■■■ * OrderItem
Product 1 ■■■ * OrderItem
Order 1 ■■■ 1 Payment
Product 1 ■■■ 1 Inventory
```

Important Order Data

```
orders
- id
- user_id
- total_amount
- order_status
- shipping_address_id
- created_at

order_items
- id
- order_id
- product_id
- quantity
- price
- subtotal
```

Store the purchased price in order_items. This preserves the price paid at the time of purchase even if the product price changes later.

4. Authentication & Authorization

Implement this before the product module.

```
POST /api/auth/register
POST /api/auth/login
```

Login should return a JWT and enough user information for the client to know the authenticated role.

```
{
  "token": "JWT_TOKEN",
  "userId": 101,
  "role": "CUSTOMER"
}
```

Example access rules:

```
GET    /api/products          → Public
POST   /api/products      → ADMIN
DELETE /api/products/{id} → ADMIN
POST   /api/orders      → CUSTOMER
GET    /api/orders      → CUSTOMER
PUT    /api/admin/orders/{id} → ADMIN
```

5. Product & Category Module

Category APIs

```
POST    /api/categories
GET     /api/categories
PUT     /api/categories/{id}
DELETE  /api/categories/{id}
```

Product APIs

```
POST    /api/products
GET     /api/products
GET     /api/products/{id}
PUT     /api/products/{id}
DELETE  /api/products/{id}
```

Product should contain at least: name, description, price, category, stock/inventory reference, image URL, active status, and timestamps.

Add search, category filtering, price filtering, sorting, and pagination after basic CRUD works.

6. Cart Module

```
POST    /api/cart/items
GET     /api/cart
PUT     /api/cart/items/{id}
DELETE  /api/cart/items/{id}
```

Business rule: if available stock is 3, the customer must not be allowed to add quantity 10. Return a clear `InsufficientStockException` or equivalent API error.

7. Address & Checkout

```
POST    /api/addresses
GET     /api/addresses
PUT     /api/addresses/{id}
DELETE  /api/addresses/{id}
```

Checkout should validate the selected address, product availability, quantities, prices, and total before creating the order.

8. Order Management

Order lifecycle:

PENDING → CONFIRMED → PROCESSING → SHIPPED → DELIVERED
■
■■■■■■■■■■■■■■■■■■■■→ CANCELLED (when eligible)

Customer APIs

```
POST /api/orders
GET  /api/orders
GET  /api/orders/{id}
PUT  /api/orders/{id}/cancel
```

Admin API

```
PUT /api/admin/orders/{id}/status
```

Order creation workflow:

```
Cart
↓
Validate stock
↓
Calculate total
↓
Create Order
↓
Create Order Items
↓
Update Inventory
↓
Create/Start Payment
↓
Clear Cart
↓
Confirm order when payment succeeds
```

9. Inventory Management

A more realistic design keeps inventory separate from product data. Track at least availableQuantity and reservedQuantity.

```
Order placed
↓
Reserve / reduce available stock
↓
Payment success
↓
Confirm inventory movement

Order cancelled/refunded
↓
Return appropriate quantity to inventory
```

10. Payment Module

Start with payment simulation. Integrate a sandbox gateway only after the complete order workflow works.

```
payments
- id
- order_id
- amount
- payment_method
- payment_status
- transaction_id
- created_at

Payment status:
PENDING
SUCCESS
FAILED
REFUNDED
```

Suggested first implementation: create a payment record, simulate success/failure, update payment status, and confirm/cancel the order consistently.

11. Transaction Management

Order creation is a multi-step business operation. Use a service-level transaction so related database operations succeed or roll back together.

```
@Transactional
public Order placeOrder(...) {
    // validate stock
    // create order
    // create order items
    // update inventory
    // clear cart
    // create payment record
```

```
}
```

This is a key interview concept: if inventory update succeeds but a later database operation fails, the transaction should prevent partially completed data from being committed.

12. Validation & Exception Handling

Use Bean Validation annotations such as `@NotBlank`, `@NotNull`, `@Email`, `@Size`, and `@Min`. Create custom exceptions for business cases.

- `ResourceNotFoundException`
- `InvalidRequestException`
- `InsufficientStockException`
- `UnauthorizedException`
- `DuplicateResourceException`

```
{
  "timestamp": "2026-08-15T10:30:00",
  "status": 400,
  "message": "Insufficient stock",
  "path": "/api/orders"
}
```

13. React Frontend

Customer pages:

```
/
■■■ Home
■■■ Login
■■■ Register
■■■ Products
■■■ Product Details
■■■ Cart
■■■ Checkout
■■■ Payment
■■■ My Orders
■■■ Order Details
■■■ Profile
■■■ Addresses
```

Admin pages:

```
/admin
■■■ Dashboard
■■■ Products
■■■ Categories
■■■ Orders
■■■ Inventory
■■■ Users
```

Use protected routes for authenticated pages and role checks for admin pages. Use Axios for API communication and a consistent frontend strategy for authentication state.

14. Admin Dashboard

- Total users
- Total products
- Total orders
- Total revenue
- Pending orders
- Low-stock products
- Recent orders

15. Testing Strategy

Postman testing:

- Authentication: registration, valid/invalid login, unauthorized access, role restrictions.
- Products: CRUD, search, filters, pagination.
- Cart: add, update, remove, and insufficient stock.
- Orders: create, retrieve, cancel, and status transitions.
- Payments: success, failure, and refund scenarios.

Automated testing: Use JUnit, Mockito, and Spring Boot Test for OrderService, PaymentService, InventoryService, and AuthenticationService. Prioritize business rules over trivial getters/setters.

16. Git & GitHub Workflow

```
Initial Spring Boot setup
Add authentication module
Add JWT security
Add product module
Add category module
Add cart module
Add order management
Add inventory management
Add payment module
Add React frontend
Add admin dashboard
Add exception handling
Add tests
Update README
```

Keep commits small and meaningful. Do not commit passwords, JWT secrets, database credentials, or payment keys.

17. Deployment

```
React frontend
  ↓
Frontend hosting

Spring Boot backend
  ↓
Backend hosting

PostgreSQL
  ↓
Cloud database
```

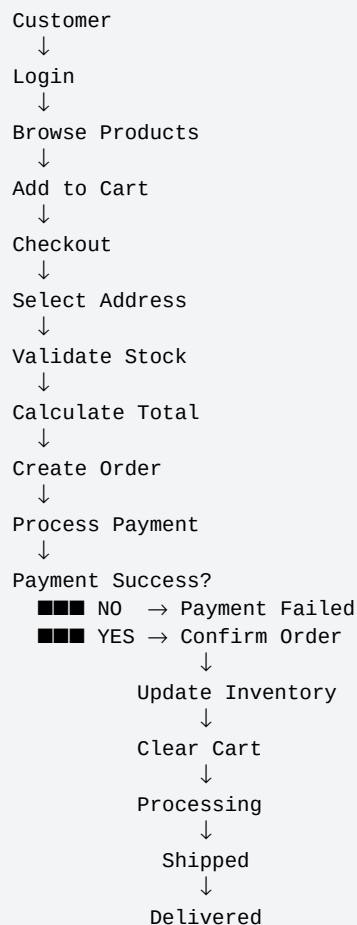
After local completion, add Docker as a portfolio enhancement. A typical containerized setup can include React, Spring Boot, and PostgreSQL.

18. Recommended Development Stages

Stage	Focus	Main Deliverables
1	Foundation	Project setup, ER design, entities, Git repository
2	Security	Users, roles, registration, login, JWT, RBAC
3	Products	Categories, product CRUD, search, filters, pagination
4	Cart	Cart items, quantities, stock validation
5	Orders	Addresses, checkout, orders, order items, status
6	Inventory & Payment	Inventory rules, transactions, payment workflow
7	React	Auth, products, cart, checkout, orders, profile
8	Admin	Dashboard, products, orders, inventory, users

Stage	Focus	Main Deliverables
9	Quality	Validation, exceptions, JUnit, Mockito, Postman
10	Release	GitHub README, Docker, deployment, final testing

19. Critical Interview Flow



You should be able to explain this flow, the entity relationships, the transaction boundary, JWT security, and the reason for separating Controller/Service/Repository layers in an interview.

20. Resume-Ready Project Positioning

Project: E-Commerce Order and Payment Management System

Role: Java Full Stack Developer

Suggested resume bullets:

- Developed an e-commerce platform for managing products, carts, orders, inventory, and payment workflows.
- Implemented JWT authentication, role-based authorization, and RESTful APIs using Spring Boot and Spring Security.
- Integrated PostgreSQL using Spring Data JPA and developed React.js interfaces for products, carts, orders, and administration.

Tech Stack: Java, Spring Boot, Spring Security, JWT, Hibernate, Spring Data JPA, React.js, PostgreSQL, REST APIs, Postman

21. Development Rules to Follow

- Do not start with payment integration. Finish authentication, products, cart, and order flow first.
- Do not put business logic in controllers.
- Use DTOs for API request/response models rather than exposing entities directly.

- Use `@Transactional` around appropriate multi-step service operations.
- Validate stock on the server; never trust frontend quantity or price.
- Do not trust price values sent by React. Fetch authoritative product prices from the database.
- Use consistent HTTP status codes and a global exception handler.
- Never hard-code database passwords, JWT secrets, or payment credentials.
- Write documentation as you build instead of waiting until the end.
- Implement features you can explain in an interview; do not add technologies only to increase the tech-stack list.